

Prova Técnica — Desenvolvimento de Sistemas

Front-end com API Externa e Banco de Dados · Stack: HTML · CSS · JavaScript · PokéAPI · Neon.tech

Tema G

Jogo de Memória Pokémon

Data: __/__/____

DURAÇÃO

3 horas

PONTUAÇÃO TOTAL

100 pontos

NOTA PARA APROVAÇÃO

60 pontos

CONSULTA

MDN, PokéAPI Docs e Neon Docs

Nome: _____

Turma: _____

Nota: _____



CONTEXTO E MISSÃO

O **Pokémon Challenge Festival** chegou à sua cidade e a organização quer digitalizar o tradicional jogo da memória com cartas de papel. A ideia é simples: o participante informa seu nome, o jogo busca Pokémon reais da internet, o cronômetro começa e — ao final — a pontuação é salva no placar global para que todos possam ver quem é o Mestre Pokémon mais rápido.

Você foi designado como o **Desenvolvedor Front-end** responsável por criar o sistema completo: a **tela inicial** (coleta o nome do jogador), o **tabuleiro do jogo** (cartas com dados reais da *PokéAPI*, lógica de par, timer e pontuação) e a

tela final com leaderboard global salvo no banco de dados via **Neon.tech**. Tudo em uma única página, sem recarregar.

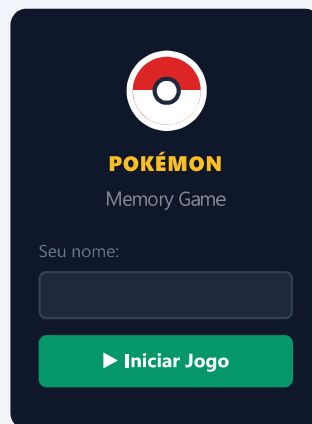
Você possui como referência o projeto do *Jogo de Memória Rick & Morty* já desenvolvido em aula. Reutilize o conhecimento de lógica de cartas, fetch de API e manipulação de DOM — e agora vá além integrando o banco de dados e o sistema de pontuação por tempo.



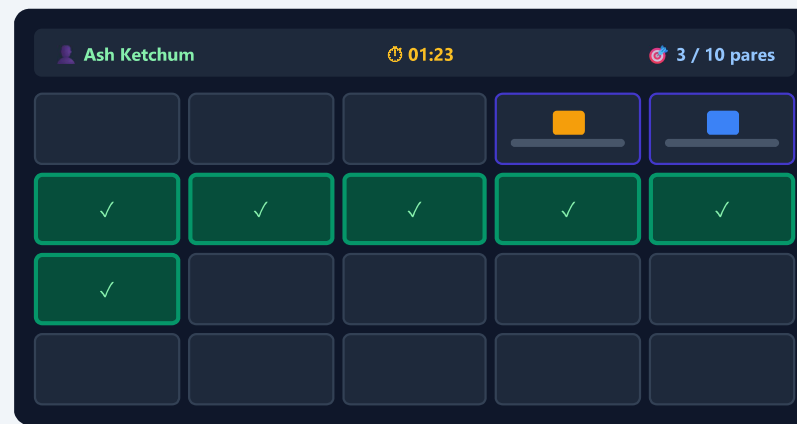
WIREFRAME DAS 3 TELAS

◆ FLUXO ESPERADO: TELA INICIAL → TELA DO JOGO → TELA FINAL (LEADERBOARD)

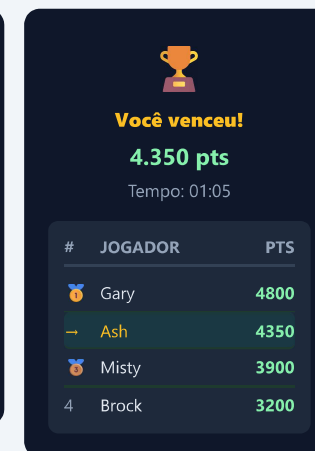
① TELA INICIAL



② TELA DO JOGO — GRADE 5 × 4 = 20 CARTAS



③ TELA FINAL + LEADERBOARD



↑ A tela inicial e a tela final são **overlays** (sobrepostos ao tabuleiro). O jogo tem sempre **20 cartas** (10 Pokémon × 2). O jogador atual é destacado com outra cor no leaderboard.



ANATOMIA DO CARD E DADOS DA POKÉAPI

Cada card possui **frente** (dados do Pokémon) e **verso** (padrão escondido). A virada é animada com CSS 3D. Os dados vêm do endpoint: <https://pokeapi.co/api/v2/pokemon/{id}>

← BACK VERSO (CARD-VERSO) — PADRÃO ESCONDIDO

Fundo escuro (#0f172a) com um símbolo de Pokébola centralizado ou padrão decorativo. O verso é a face padrão ao iniciar o jogo — sem dados do Pokémon. CSS: `backface-visibility: hidden`.

🔴 FRENTE (CARD-FRENTE) — DADOS DO POKÉMON

3 elementos obrigatórios:

- `data.sprites.other['official-artwork'].front_default` → imagem
- `data.name` → nome do Pokémon
- `data.types[0].type.name` → tipo principal

O tipo é exibido como uma badge colorida (`.badge-tipo`). CSS: `transform: rotateY(180deg)`.

Exemplo de resposta da PokéAPI e extração dos campos:

```
// Fetch de um Pokémon pelo ID
const resposta = await fetch(`https://pokeapi.co/api/v2/pokemon/${idAleatorio}`);
const dados = await resposta.json();

const pokemon = {
  id:      dados.id,
  nome:    dados.name,
  imagem:  dados.sprites.other['official-artwork'].front_default,
  tipo:    dados.types[0].type.name // "fire", "water", "grass"...
};
```

Badges de tipo sugeridas — use as classes CSS a seguir (ou adapte com a paleta):

fire **water** **grass** **psychic / ghost** **electric** **normal / outros**

```
/* Estrutura CSS obrigatória para o flip 3D */
.card { perspective: 1000px; cursor: pointer; }
.card-inner { position: relative; width: 100%; height: 100%; transform-style: preserve-3d; transition:
transform 0.5s ease; }
```

```
.card.card-virada .card-inner { transform: rotateY(180deg); }  
.card-frente, .card-verso { position: absolute; width: 100%; height: 100%; backface-visibility:  
hidden; }  
.card-frente { transform: rotateY(180deg); }  
.card.card-acertada { pointer-events: none; opacity: 0.85; border-color: var(--emerald-600); }
```



PALETA DE CORES OFICIAL

Declare `:root {}` no seu CSS. O design do jogo pode usar cores extras (ex: #0f172a para o fundo escuro), mas os componentes de interface (HUD, leaderboard, toasts) devem usar as variáveis abaixo.



DADOS DO BANCO DE DADOS

O banco já está criado. O professor fornecerá o endpoint e o token antes do início.

CAMPO	VALOR
Endpoint HTTP (Neon)	https://_____.neon.tech/sql/v1/query
Token (Bearer)	_____
Tabela	ranking

CAMPO	VALOR
Colunas	<code>id</code> (serial PK) · <code>nome_jogador</code> (varchar) · <code>pontuacao</code> (integer) · <code>tempo_segundos</code> (integer) · <code>jogado_em</code> (timestamp DEFAULT NOW() — gerado pelo banco)

FÓRMULA DE PONTUAÇÃO

A pontuação é inversamente proporcional ao tempo: quanto mais rápido o jogador completa o jogo, mais pontos ganha. Use exatamente a fórmula abaixo — ela deve ser implementada no momento em que `verificarVitoria()` detecta que todos os pares foram encontrados.

```
// Fórmula obrigatória — tempoDecorrido em segundos inteiros
const pontuacao = Math.max(0, Math.floor(5000 - (tempoDecorrido * 10)));

// Exemplos:
// 30s → 5000 - 300 = 4.700 pts (muito rápido)
// 64s → 5000 - 640 = 4.360 pts (bom tempo)
// 200s → 5000 - 2000 = 3.000 pts (tempo moderado)
// 500s → Math.max(0, ...) = 0 pts (proteção contra negativo)
```

💡 O tempo formatado para exibição na tela usa `formatarTempo(segundos)`: `const min = Math.floor(s/60).toString().padStart(2, '0');` `const sec = (s%60).toString().padStart(2, '0');` `return `${min}:${sec}`;`

TAREFAS E REQUISITOS

O projeto deve ser organizado em **4 arquivos separados**:

ARQUIVO	RESPONSABILIDADE
<code>index.html</code>	Estrutura das 3 telas (overlays de inicial e final + grade do jogo), HUD, toast container
<code>style.css</code>	Estilos, variáveis CSS, animação flip 3D, badges de tipo, leaderboard, toasts
<code>script_db.js</code>	Motor de fetch Neon + funções <code>salvarPontuacao</code> e <code>buscarLeaderboard</code> (exportadas)
<code>script.js</code>	Toda a lógica do jogo: API Pokémon, timer, matching de cartas, pontuação, eventos

O `script.js` deve ser carregado com `type="module"` e importar as funções do `script_db.js`.

Tela Inicial (`#tela-inicial`): overlay centralizado, visível ao carregar. Deve conter:

- Logo/título do jogo (estilizado)
- Campo `<input type="text" id="input-nome">` — nome do jogador (obrigatório)
- Botão "Iniciar Jogo" que dispara `iniciarJogo()`

HUD (`#hud`) — visível durante o jogo:

- `#hud-nome`: exibe o nome do jogador
- `#hud-timer`: exibe o tempo no formato `MM:SS`
- `#hud-pares`: exibe pares encontrados / total (ex: "3 / 10")

Grade de cartas (`#grid-cartas`): CSS Grid com **5 colunas** — comporta 20 cartas (4 linhas × 5 colunas). Cada carta gerada pelo JS deve ter:

- Estrutura com `.card`, `.card-inner`, `.card-verso` e `.card-frente`

- Atributo `data-key` com o ID do Pokémon (para comparação de pares)
- Animação CSS 3D obrigatória conforme especificado na seção "Anatomia do Card"

Tela Final (`#tela-final`): overlay oculto por padrão. Deve conter:

- Pontuação final e tempo formatado do jogador
- Tabela de leaderboard gerada dinamicamente (`#tabela-ranking`)
- Botão "Jogar Novamente" que dispara `reiniciarJogo()`

🚫 **Proibido:** frameworks CSS (Bootstrap, Tailwind etc.) e frameworks JS (React, Vue etc.). CSS3 e JavaScript puro. A animação de flip deve usar `transform: rotateY(180deg)` — não usar `display:none` para esconder a frente.

2

PokéAPI — Busca, Extração e Renderização dos Cards

Fetch assíncrono · IDs únicos aleatórios · Duplicação e embaralhamento

25 pontos

No `script.js`, implemente a função `buscarPokemonsAleatorios(quantidade)` que:

- Gera `quantidade` IDs **únicos e aleatórios** entre 1 e 151 (Geração I). Use um `Set` para garantir unicidade:

```
async function buscarPokemonsAleatorios(quantidade) {
  const ids = new Set();
  while (ids.size < quantidade) {
    ids.add(Math.floor(Math.random() * 151) + 1);
  }
  const pokemons = [];
  for (const id of ids) {
    const resp = await fetch(`https://pokeapi.co/api/v2/pokemon/${id}`);
    const dados = await resp.json();
    pokemons.push({ id: dados.id, nome: dados.name,
      imagem: dados.sprites.other['official-artwork'].front_default,
```

```
    tipo: dados.types[0].type.name });  
  }  
  return pokemons;  
}
```

Implemente a função `renderizarCartas(pokemons)` que:

- Duplica o array e embaralha: `[...pokemons, ...pokemons].sort(() => Math.random() - 0.5)`
- Limpa o `#grid-cartas` antes de renderizar
- Para cada Pokémon, cria um elemento `.card` com a estrutura flip (frente + verso) e insere no grid
- A frente exibe: `<img>` com a imagem oficial, `<p>` com o nome e `<span class="badge-tipo">` com o tipo
- O atributo `data-key` recebe o `id` do Pokémon — mesmo valor nas duas cartas do par

💡 A imagem `official-artwork` pode demorar para carregar. Adicione `loading="lazy"` na tag `<img>` e um placeholder de fundo no CSS enquanto carrega. O `alt` deve conter o nome do Pokémon.

🚫 **Proibido** usar listas fixas/hardcoded de Pokémon. O fetch à PokéAPI é obrigatório — assim como ocorreu com a API do Rick & Morty na referência. Projetos sem fetch real à PokéAPI terão nota zero na Tarefa 2.

3

Lógica do Jogo — Timer, Matching e Pontuação

iniciarJogo · setInterval · flag bloqueado · verificarVitoria

25 pontos

Variáveis de estado — declare no escopo global do `script.js`:

```
let carta1 = null;      // 1ª carta clicada  
let carta2 = null;      // 2ª carta clicada  
let bloqueado = false;  // impede cliques durante animação
```

```
let paresEncontrados = 0; // contador de pares certos
const totalPares = 10; // total de pares no jogo
let timer = null; // referência do setInterval
let tempoDecorrido = 0; // segundos desde o início
let nomeJogador = ''; // capturado na tela inicial
```

Função `iniciarJogo()` — chamada pelo botão da tela inicial:

- Valida que o campo de nome não está vazio. Se vazio: exibir toast de aviso e não prosseguir.
- Armazena `nomeJogador = document.getElementById('input-nome').value.trim()`
- Esconde a `#tela-inicial` (adicionar classe `hidden`)
- Exibe o HUD com o nome do jogador em `#hud-nome`
- Inicia o cronômetro com `setInterval` a cada 1000ms: incrementa `tempoDecorrido` e atualiza `#hud-timer` com `formatarTempo(tempoDecorrido)`
- Chama `buscarPokemonsAleatorios(10)` e, ao receber os dados, chama `renderizarCartas(pokemons)`
- Adiciona o listener de clique no `#grid-cartas` usando event delegation

Lógica de clique nas cartas (handler dentro de `iniciarJogo`):

- Se `bloqueado === true` ou a carta já tem classe `card-acertada` ou `card-virada`: **ignorar o clique**
- Adiciona `card-virada` na carta clicada
- Se `carta1 === null`: armazena como primeira carta e aguarda
- Se já existe `carta1`: armazena como `carta2`, define `bloqueado = true` e compara:

```
// Par CORRETO: mesmo data-key
if (carta1.dataset.key === carta2.dataset.key) {
  carta1.classList.add('card-acertada');
  carta2.classList.add('card-acertada');
  paresEncontrados++;
  document.getElementById('hud-pares').textContent = `${paresEncontrados} / ${totalPares}`;
```

```

    carta1 = carta2 = null;
    bloqueado = false;
    verificarVitoria();
  } else {
    // Par ERRADO: aguarda 1s e vira de volta
    setTimeout(() => {
      carta1.classList.remove('card-virada');
      carta2.classList.remove('card-virada');
      carta1 = carta2 = null;
      bloqueado = false;
    }, 1000);
  }
}

```

Função `verificarVitoria()`:

- Se `paresEncontrados === totalPares`: chama `clearInterval(timer)`
- Calcula `pontuacao` com a fórmula obrigatória
- Chama `encerrarJogo(nomeJogador, pontuacao, tempoDecorrido)`

Função `reiniciarJogo()` — botão "Jogar Novamente":

- Reseta todas as variáveis de estado para os valores iniciais
- Limpa o `#grid-cartas` e o `#tabela-ranking`
- Esconde a `#tela-final` e exibe a `#tela-inicial`

4

Banco de Dados — Salvar Pontuação e Leaderboard

executarQueryNeon · INSERT · SELECT TOP 10 · Tela final

30 pontos

No `script_db.js` — motor de conexão (idêntico ao padrão da série):

```
const DATABASE_URL = "postgresql://usuario:senha@host/banco?sslmode=require";
const host = new URL(DATABASE_URL).host;
const neonHttpEndpoint = `https://${host}/sql`;

async function executarQueryNeon(querySQL, parametros = []) {
  try {
    const r = await fetch(neonHttpEndpoint, {
      method: 'POST',
      headers: { 'Neon-Connection-String': DATABASE_URL, 'Content-Type': 'application/json' },
      body: JSON.stringify({ query: querySQL, params: parametros })
    });
    if (!r.ok) throw new Error(await r.text());
    const dados = await r.json();
    return dados.rows;
  } catch (e) { console.error(e); return null; }
}
```

Funções exportadas do `script_db.js`:

- `export async function salvarPontuacao(nome_jogador, pontuacao, tempo_segundos)` — executa `INSERT INTO ranking (nome_jogador, pontuacao, tempo_segundos) VALUES ($1, $2, $3) RETURNING *`. Retorna `true` se sucesso, `false` se erro.
- `export async function buscarLeaderboard()` — executa `SELECT * FROM ranking ORDER BY pontuacao DESC LIMIT 10`. Retorna o array de registros ou `[]`.

```
// INSERT
"INSERT INTO ranking (nome_jogador, pontuacao, tempo_segundos) VALUES ($1, $2, $3) RETURNING *"
params: [nome_jogador, pontuacao, tempo_segundos]

// SELECT leaderboard
```

```
"SELECT * FROM ranking ORDER BY pontuacao DESC LIMIT 10"
params: []
```

Função `encerrarJogo(nome, pontuacao, tempo)` no `script.js`:

- Exibe toast de sucesso com a pontuação: ex. "🎉 Você fez 4.350 pontos!"
- Chama `await salvarPontuacao(nome, pontuacao, tempo)`. Se erro: toast de erro, mas continua para mostrar o leaderboard local.

Chama `await buscarLeaderboard()` e renderiza a tabela `#tabela-ranking`:

```
// Renderizar leaderboard
const medalhas = ['🥇', '🥈', '🥉'];
ranking.forEach((r, i) => {
  const destaque = r.nome_jogador === nomeJogador ? 'class="linha-destaque"' : '';
  tbody.innerHTML += `
    <tr ${destaque}>
      <td>${medalhas[i] ?? i + 1}</td>
      <td>${r.nome_jogador}</td>
      <td>${r.pontuacao.toLocaleString('pt-BR')} pts</td>
      <td>${formatarTempo(r.tempo_segundos)}</td>
    </tr>`;
});
```

- Exibe a `#tela-final` com a pontuação e tempo do jogador atual

Toasts — idênticos ao padrão da série:

- `mostrarToast(mensagem, tipo)` com `'success'` (verde), `'error'` (rose) e `'info'` (azul)
- Some após 3 segundos com animação `@keyframes slideIn`
- Fixado no `#toast-container` (canto inferior direito)

💡 O endpoint do Neon é construído via `new URL(DATABASE_URL).host` — nunca faça hardcode do host. A função `executarQueryNeon` é local ao `script_db.js` (não exportada); apenas `salvarPontuacao` e `buscarLeaderboard` são exportadas.



CRITÉRIOS DE AVALIAÇÃO

CRITÉRIO	PESO	NOTA OBTIDA
TAREFA 1 — ESTRUTURA VISUAL (HTML + CSS)		
Projeto em 4 arquivos (index.html, style.css, script.js, script_db.js) com type="module"; 3 overlays corretos (#tela-inicial, HUD, #tela-final)	5 pts	_____
Tela inicial com campo de nome e botão; HUD com #hud-nome, #hud-timer e #hud-pares; tela final com pontuação, tempo e área do leaderboard	5 pts	_____
Grid de cartas com 5 colunas; card com estrutura .card > .card-inner > (.card-verso + .card-frente) correta; atributo data-key presente em cada carta	5 pts	_____
Animação flip 3D com rotateY(180deg), backface-visibility: hidden e transform-style: preserve-3d; variáveis CSS da paleta; classes .card-virada e .card-acertada com estilos distintos; sem frameworks	5 pts	_____
TAREFA 2 — POKÉAPI E RENDERIZAÇÃO DOS CARDS		
buscarPokemonsAleatorios(10) com Set para IDs únicos, await fetch à PokéAPI para cada ID, tratamento de erro com try/catch	8 pts	_____

CRITÉRIO	PESO	NOTA OBTIDA
Extração correta dos 3 campos obrigatórios: <code>sprites.other['official-artwork'].front_default</code> , <code>data.name</code> e <code>data.types[0].type.name</code>	5 pts	_____
Duplicação + embaralhamento corretos (<code>[...arr,...arr].sort()</code>); frente do card com imagem, nome e badge de tipo; <code>data-key</code> igual para as duas cartas do mesmo par	7 pts	_____
Funções exportadas no <code>script_db.js</code> e importadas no <code>script.js</code> com sintaxe ES6 (<code>import { ... } from</code>)	5 pts	_____
TAREFA 3 — LÓGICA DO JOGO, TIMER E PONTUAÇÃO		
<code>iniciarJogo()</code> : valida nome, esconde overlay inicial, exibe HUD, inicia <code>setInterval</code> e chama <code>buscar</code> + <code>renderizar</code>	8 pts	_____
Lógica de clique: flag bloqueado, 2 cartas por vez, comparação por <code>data-key</code> , <code>card-acertada</code> no par correto, virar de volta após 1s no par errado, atualização do <code>#hud-pares</code>	8 pts	_____
<code>verificarVitoria()</code> : detecta todos os pares, <code>clearInterval</code> , calcula pontuação com a fórmula obrigatória <code>Math.max(0, Math.floor(5000 - tempo * 10))</code> e chama <code>encerrarJogo()</code>	5 pts	_____
<code>reiniciarJogo()</code> : reseta todas variáveis de estado, limpa grid e ranking, volta à tela inicial	4 pts	_____
TAREFA 4 — BANCO DE DADOS, SALVAMENTO E LEADERBOARD		
<code>executarQueryNeon()</code> com <code>Neon-Connection-String</code> , endpoint via <code>new URL().host</code> sem <code>hardcode</code> , <code>try/catch</code> , retorna <code>dados.rows</code> ou <code>null</code>	8 pts	_____
<code>salvarPontuacao(nome, pontuacao, tempo)</code> com <code>INSERT</code> parametrizado <code>RETURNING *</code> ; retorna <code>true/false</code> ; toast de sucesso ou erro após a operação	8 pts	_____

CRITÉRIO	PESO	NOTA OBTIDA
buscarLeaderboard() com SELECT ORDER BY DESC LIMIT 10; leaderboard renderizado com posição (medalhas no top 3), nome, pontuação formatada e tempo; linha do jogador atual destacada	9 pts	_____
mostrarToast() com 3 tipos (success/error/info), cores corretas, animação @keyframes slideIn, some em 3s	5 pts	_____
TOTAL	100 pts	_____



O QUE ENTREGAR

- ✓ Os 4 arquivos: index.html, style.css, script.js e script_db.js — em **.zip** ou repositório público no GitHub.
- ✓ A aplicação será testada abrindo o index.html diretamente no navegador (sem servidor local). As requisições à PokéAPI e ao Neon devem funcionar via fetch real.
- ✓ Gravação de tela demonstrando o fluxo completo: **tela inicial** (digitar nome) → **carregamento dos Pokémon** → **jogo com ao menos 2 pares errados visíveis** → **vitória** → **leaderboard** com a pontuação gravada. As 20 cartas devem estar visíveis durante o jogo.
- ✓ Na gravação, executar o jogo ao menos **duas vezes** para que o leaderboard mostre múltiplos registros no banco.

⚠ **Nota zero nas Tarefas 2, 3 e 4** para projetos que usem listas fixas de Pokémon hardcoded, localStorage como substituto do banco, ou que não executem fetch real à PokéAPI e ao Neon.

🔗 **Dica de organização:** comece pela Tarefa 1 (estrutura HTML e CSS flip) → Tarefa 2 (teste o fetch da PokéAPI no console antes de integrar) → Tarefa 3 (jogo funcional sem banco) → Tarefa 4 (integração com Neon). Ter o jogo jogável antes de conectar

ao banco facilita o debug.

Dúvidas sobre fetch Neon ou PokéAPI → pergunte ao professor/avaliador.

Lógica do jogo, queries SQL e extração dos dados da API são de inteira responsabilidade do aluno.

Gotta code 'em all! 🎮